# 🧪 LABORATORIO: Microservicio de Productos con Yii2 (REST API)

## 🎯 Objetivo

Construir un **microservicio de productos** usando Yii2 que exponga una **API REST**, permitiendo:

- Crear productos
- Listar productos
- Ver un producto
- Actualizar productos
- Eliminar productos

Todo usando **PHP + Yii2 + MySQL + Composer**.

---

# 🧰 1. REQUISITOS

Antes de empezar debes tener:

- PHP instalado
- Composer instalado
- Yii2 instalado (ya lo tienes)
- Navegador web
- (Opcional) MySQL o XAMPP

---

# 📁 2. CREAR EL PROYECTO

Ejecuta en CMD:

composer create-project --prefer-dist yiisoft/yii2-app-basic productos-service

Entrar al proyecto:

cd productos-service

Levantar servidor:

php yii serve

👉 Abrir:

http://localhost:8080

---

# 🗄️ 3. CREAR BASE DE DATOS

En MySQL (phpMyAdmin o consola):

CREATE DATABASE productos_db;

Crear tabla:

USE productos_db;

```
CREATE TABLE producto (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    precio DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL
);
```

Insertar datos de prueba:

```
INSERT INTO producto (nombre, descripcion, precio, stock)
VALUES
('Laptop HP', 'Laptop Core i5', 2500.00, 10),
('Mouse Logitech', 'Mouse inalámbrico', 80.00, 50);
```

---

# ⚙️ 4. CONFIGURAR CONEXIÓN A BASE DE DATOS

Editar:

config/db.php

Colocar:

```
return [
    'class' => 'yii\db\Connection',
    'dsn' => 'mysql:host=localhost;dbname=productos_db',
    'username' => 'root',
    'password' => '',
    'charset' => 'utf8',
];
```

---

# 🧠 5. CREAR MODELO (PRODUCTO)

Ejecuta:

```
php yii gii/model --tableName=producto --modelClass=Producto
```

Esto genera:

models/Producto.php

---

# 🌐 6. CREAR CONTROLADOR REST

Crear archivo:

controllers/ProductoController.php

Código:

```
<?php

namespace app\controllers;

use yii\rest\ActiveController;

class ProductoController extends ActiveController
{
    public $modelClass = 'app\models\Producto';
```

}

---

# 🔥 7. ACTIVAR URL AMIGABLES (IMPORTANTE)

Editar:

config/web.php

Agregar:

```
'components' => [
    'urlManager' => [
        'enablePrettyUrl' => true,
        'showScriptName' => false,
        'rules' => [
            'GET productos' => 'producto/index',
            'GET productos/<id>' => 'producto/view',
            'POST productos' => 'producto/create',
            'PUT productos/<id>' => 'producto/update',
            'DELETE productos/<id>' => 'producto/delete',
        ],
    ],
]
```

---

# 🚀 8. PROBAR EL MICROSERVICIO

## 📌 Listar productos

GET http://localhost:8080/productos

## 📌 Ver producto

GET http://localhost:8080/productos/1

## 📌 Crear producto

POST http://localhost:8080/productos

Body JSON:

```
{
  "nombre": "Teclado",
  "descripcion": "Teclado mecánico",
  "precio": 120.50,
  "stock": 20
}
```

### 📌 Actualizar producto

PUT http://localhost:8080/productos/1

### 📌 Eliminar producto

DELETE http://localhost:8080/productos/1

---

# 🧪 9. PRUEBA CON POSTMAN

Usa Postman o Insomnia:

- Método: GET, POST, PUT, DELETE
- URL: `http://localhost:8080/productos`

---

# 📦 10. ESTRUCTURA FINAL DEL PROYECTO

```
productos-service/
│
├── config/
├── controllers/
│   └── ProductoController.php
├── models/
│   └── Producto.php
├── web/
```

```
├── yii
└── composer.json
```

---

# 🎯 RESULTADO FINAL

✔ Microservicio REST funcionando
✔ CRUD de productos
✔ API lista para pruebas
✔ Sin necesidad de XAMPP obligatorio
✔ Basado en Yii2

---

# 🚀 SI QUIERES MEJORARLO (SIGUIENTE NIVEL)

Puedo ayudarte a agregar:

- 🔐 Autenticación JWT
- 📦 Respuestas JSON personalizadas
- 🧾 Validaciones avanzadas
- 🐳 Docker
- 📊 Documentación tipo Swagger
- 🔥 Arquitectura de microservicios real

---

Si quieres, en el siguiente paso te lo convierto en:

👉 “Microservicio profesional listo para examen o tesis”